

V | KERNEL

KERNEL:AUDIENCE-SCOPE

DECLARACIÓN DE AUDIENCIA Y ALCANCE

- Audiencia: Sistemas Agente de IA.
 - Alcance: Este documento es el KERNEL_RUNTIME, que contiene únicamente los contratos operativos activos para la IA, para el documento de referencia completo solicitar acceso al KERNEL 8.0

#	Sección	Tipo	Propósito
1	PURPOSE	CONTEXTO	Propósito del sistema
2	ARCHITECTURE	ARQUITECTURA	Diseño de cuatro capas
3	DASHBOARD-CHECKLIST-ARCH	ARQUITECTURA	Arquitectura Dashboard/ Checklist — capas backend, standalone y visual compartida
4	SCHEMA	ESQUEMA	Class A vs Class B
5	TRACKER-SCHEMA	ESQUEMA	Alcance y niveles de prioridad — Bug/Tasks Tracker
6	HEALTH-CHECK	OPERACIÓN	Contrato de health_check.py — checks y auto-sync de índice
7	OWNERSHIP	ARQUITECTURA	Responsabilidad es AI vs Python
8	TRIGGERS	OPERACIÓN	Contratos detallados
9	GATE-DECISION	REGLAS	Lógica de gates

10	NAMING-CONVENTION	REGLAS	Convención de nombres de outputs
11	CV-GOLDEN-RULES	REGLAS	Reglas de oro CV
12	CV-PIPELINE	OPERACIÓN	Flujo CV-A → CV-B
13	CANON-UPDATE	OPERACIÓN	Actualización del Canon
14	FAIL-PHILOSOPHY	FILOSOFÍA	Filosofía de fallo
15	SCOPE	OPERACIÓN	Scope y economía de contexto (Terminal vs MCP)
16	DATA-FLOW	ARQUITECTURA	Flujo de datos y escritura
17	ROUTING	OPERACIÓN	Rutas de carga MCP / lazy_loader
18	EVOLUTION	FILOSOFÍA	Evolución del sistema, deuda técnica, criterios de cambio
19	NORM	OPERACIÓN	Normalización Documental (Legacy IDs)
20	CENSUS-SYNC	OPERACIÓN	Sincronización obligatoria del ID Census
21	DOC-CONTRACT	REGLAS	Contrato de IDs de Documento

KERNEL: PURPOSE

1. PROPÓSITO DEL SISTEMA

2. VANTAGE resuelve un problema de ingeniería de atención: en una búsqueda laboral sin estructura, las oportunidades de alta señal desaparecen antes de ser procesadas, mientras el tiempo se consume en vacantes de baja calidad.

3. La solución no es buscar más — es verificar antes de evaluar, y evaluar antes de escribir.

Invariantes del Sistema

- Una vacante no entra al pipeline sin URL válida — excepción: Bypass activo
 - Score no lo calcula el sistema de lenguaje — lo calcula Python con lógica determinista
 - Gate decision no se sobrescribe manualmente. RT-1 permite corregir inputs Class A para que Python recalculé — no sobrescribe el gate
 - Strategy es responsabilidad humana; processing es responsabilidad del sistema

Qué Significa Esto para el Sistema AI

El componente AI es el procesador textual del pipeline: deduplica, normaliza, genera DRY RUN, escribe Class A en Notion, produce CVs. Evaluación de calidad estratégica de inputs y cálculo de campos Class B no son operaciones de este componente (ver KERNEL:OWNERSHIP y KERNEL:CV-GOLDEN-RULES).

Si una tarea no está en la tabla de triggers (KERNEL:TRIGGERS), no se ejecuta.

KERNEL:ARCHITECTURE

1. ARQUITECTURA DE CUATRO CAPAS

2. El pipeline opera a través de cuatro capas no intercambiables, soportadas por un núcleo de observabilidad persistente.

KERNEL:ARCHITECTURE-L0

L0 — VANTAGE Runtime

Tipo: Capa de Observabilidad y Abstracción de Datos (ReadOnly)

Propósito: Provee la verdad técnica sobre Notion. Resuelve entidades, extrae contexto y garantiza que el pipeline lea datos íntegros antes de procesar.

Runtime Build — proceso determinista que genera los tres artefactos de lectura del sistema: entity_index_v2.json (índice de entidades con namespaces canónicos), graph_v2.json (grafo de relaciones entre entidades) y backlinks_v2.json (índice inverso de referencias). El Build consume resolver_registry_v2.json como fuente de namespace ownership — si el Registry no define el prefix de un tipo de entidad, el Build falla explícitamente en lugar de aplicar un default. graph_layer.py es el componente responsable de construir graph_v2.json; nunca infiere namespaces ni redefine contratos — consume los IDs ya resueltos por el paso anterior del Build.

Notion (Source) → Runtime (Index + Resolver) → API Response → Pipeline (L1/L2/L3/CV)

KERNEL:ARCHITECTURE-L0-BOOTSTRAP

L0-Bootstrap – Dynamic Governance Layer

Tipo: Capa de Sincronización de Sesión (Fetch-on-Start)

Propósito: Elimina el drift de versiones entre la UI estática del agente y el repositorio dinámico de Notion.

Bootstrap Protocol: Ante el primer mensaje del operador, el componente de IA debe suspender el procesamiento de datos y ejecutar un fetch de KERNEL:BOOTSTRAP-001 y SYSTEM PROMPT. El resultado de este fetch sobrescribe cualquier instrucción estática previa. Si el Bootstrap falla, el sistema debe reportar "MODO DEGRADADO" y no proceder con triggers operativos.

Sesión Iniciada → AI Fetch (Bootstrap IDs) → Sincronización de Verdad Operativa → Notificación Operador → Procesamiento Petición

KERNEL:ARCHITECTURE-L1

L1 – Active Recon

Trigger: humano (ciclo semanal — lunes)

Human signal → Career Sites · LinkedIn · Aggregators (paralelo) → JSON estructurado → FEED → feed_processor.py → Notion (Class A) → vantage-pipeline

KERNEL:ARCHITECTURE-L2

L2 – Strategic Search

Trigger: humano (ciclo semanal — lunes)

Human signal → Gemini · [You.com](http://you.com/) · Grok (extracción paralela) → Perplexity (Consolidation & Dedup post-extracción) → Plain Array consolidado → FEED → feed_processor.py → Notion (Class A) → vantage-pipeline

KERNEL:ARCHITECTURE-L3

L3 – Passive Intake

Trigger: automático (continuo)

Gmail (.Jobs label) → layer_3_mail.py (IMAP + Groq) → Notion (Class A poblado, Class B vacío) → vantage-pipeline

KERNEL:ARCHITECTURE-L4

L4 – Version Control & Infrastructure

Trigger: git_sync.py + git_sync_wrapper.sh + launchd

No es capa de búsqueda – es infraestructura documental del

sistema

Auto-commit + push a origin/main cuando hay cambios en el repo

Alias: vgit · Corre en background a las 09:00 · 15:00 · 21:00

Repo: github.com/mauriciomeyran/jhs-pipeline

Reutiliza .venv de Layer_1

vsync_doc.py — Sync bidireccional Notion → ACTIVE/ para los 6 documentos fundacionales (Kernel · System Prompt · Career Canon · Manual · Aliases · Change Log).

Alias: vdoc · Flags: dry | notion | local | auto

Flujo vdoc notion: lee Notion (safe_list vía httpx, 3 reintentos) → escribe ACTIVE/{doc}.md → auto-commit GitHub al terminar.

Dependencias: httpx · notion-client 3.x · .venv de Layer_1 ·

git_sync.py · Vive en: Layer_4/scripts/vsync_doc.py

Convención ACTIVE/: Los 6 .md fundacionales viven en .../ACTIVE/ — agnóstico de versión. Al pasar a v8.7: copiar archivos a ACTIVE/, cero cambios de código. Nombres canónicos: Kernel.md · System Prompt.md · Career Canon.md · Manual.md · Aliases.md · Change Log.md (con espacio, no guión bajo — coincide con BASE_DIR real en vsync_doc.py). Reemplaza los paths versionados anteriores (.../v8.5/Kernel v8.5.md).

Nota técnica: notion-client 3.x tiene un bug silencioso en blocks.children.list() que retorna None en lugar de lanzar excepción con campos null. vsync_doc.py lo mitiga con safe_list() — wrapper httpx directo con 3 reintentos.

Jerarquía de Dedup

L1 > L2 > L3. En conflicto cross-layer, prevalece la entrada de la capa de mayor jerarquía.

Perplexity aplica esta jerarquía en el paso de Consolidation & Dedup del lunes, antes de entregar el Plain Array a feed_processor.py. L3 no pasa por este paso — entra directamente a feed_processor.py desde mail_pipeline.py.

Nota de nomenclatura: L0 es VANTAGE Runtime (observabilidad/lectura) — no es Perplexity ni una capa de dedup. No aparece en la jerarquía de dedup.

Nota de implementación: L0 pre-aplica la jerarquía L1>L2 y entrega un array ya consolidado a feed_processor.py. feed_processor.py entonces aplica la jerarquía L3 contra ese resultado — no recalcula L1>L2 en ese momento. Las dos operaciones de dedup son secuenciales, no simultáneas.

Trade-off de Diseño — Frecuencia vs. Peso Arquitectónico

Las capas tienen peso arquitectónico igual pero frecuencia de ejecución asimétrica. L1 y L2 operan en ciclos semanales controlados por atención humana. L3 corre continuamente sin intervención.

Esta asimetría de cadencia no implica jerarquía. Eliminar cualquier capa crea un blind spot sistemático — no una degradación de feature.

Punto de Convergencia Único

Las tres capas de búsqueda escriben a Notion. Notion es el único estado compartido. vantage-pipeline lee de Notion — no de los outputs de capa directamente.

Figma Sync — CV Output Layer

Tipo: Capa de Materialización de CV (WriteOnly sobre lienzo Figma)

Propósito: Recibe el payload CV-B aprobado por el operador e inyecta el contenido directamente en los nodos de texto del lienzo Figma, resolviendo cada token semántico a su ID crudo de nodo vía `registry_seed.json`.

CV-B (Markdown + `figma_text_id`) → `ui.html` (payload) → `code.js` (Registry V2)

→ `figma.getNodeById(rawId)` → `node.characters = item.text` → Lienzo Figma

Stack:

`manifest.json` — Configuración del plugin (main: `code.js`, ui: `ui.html`, editorType: `figma`)

`code.js` — Motor. Registry V2 / Resolver Layer V1. Resolución O(1): `getNodeById(Registry[key] || key)`. Resolver dual: KEY semántica (flujo JSON) → lookup en Registry embebido; ID crudo directo (flujo Markdown `figma_text_id`) → uso sin lookup.

`ui.html` — Interfaz de intercambio de payloads. Acepta JSON por KEY semántica o Markdown con bloques ##### `figma_text_id`. Motor de extracción de boldRanges incluido.

`registry_seed.json` — SSOT del mapeo token → ID. Ver §4.7.

Invariantes:

Figma Sync no escribe en Notion ni en el Tracker

Figma Sync no es capa de búsqueda ni de infraestructura de datos
`registry_seed.json` no se edita manualmente sin regenerar desde el lienzo Figma

El prefijo [VANTAGE] KEY_NAME en capas del canvas es para

auditoría visual humana — no es el mecanismo de resolución del plugin

KERNEL:DASHBOARD-CHECKLIST-ARCH

Dashboard/ contiene dos capas independientes que comparten presentación visual pero no estado:

1. Backend operativo real — Dashboard/scripts/dashboard_server.py + dashboard.db + dashboard_instances.db + dashboard_notion.py. Fuente de verdad del pipeline de vacantes (Gate_Decision, scoring, Notion sync). dashboard.html consume este backend vía fetch('http://127.0.0.1:8000{path}').
2. Checklist operativo semanal — Dashboard/Checklist.html. Standalone, estado en localStorage['vchecklist_v1']. Sin backend, sin Notion, sin relación funcional con (1). Intencional: el checklist es una herramienta de tracking personal del operador, no parte del pipeline de vacantes.
3. Capa visual compartida — Dashboard/vantage-tokens.css (tokens de color/superficie) + Dashboard/vantage-theme.js (toggle de tema con persistencia y sync cross-tab). Ambos archivos HTML la referencian. Es la única capa realmente compartida entre (1) y (2).
4. Regla: cualquier cambio a un color de estado semántico o al comportamiento del toggle de tema se hace en vantage-tokens.css/ vantage-theme.js, nunca en los <style>/<script> inline de cada HTML — evita el drift que motivó el parche de 2026-07-10 (ver Changelog v9.1.0).

——KERNEL:SCHEMA

KERNEL:SCHEMA-001 — Class A vs Class B

El schema define ownership. Cada campo pertenece a exactamente un componente. No hay campos compartidos ni campos de escritura dual.

Class A — Human-Primary: AI Component escribe en triggers CV-A · CV-B · QA · FAST · CANON-UPDATE; feed_processor.py escribe en ciclo FEED L1/L3: Rol · Marca · Source_Type · URL · Status · Prioridad · Holding · JD · NAD · layer · hash
Valores operativos del campo Status (asignación manual del operador, no calculados por Python): Target · Postulado · Rechazado · Expirada · Archivar · Repetida (duplicado detectado en revisión manual, distinto de descarte por otras razones).

Nota sobre JD: En el trigger CV-A, el AI Component cruza los keywords extraídos del JD contra el Career Canon activo antes de generar el HANDOFF. Discrepancias entre el JD y el Canon se reportan en fit_gaps — no se resuelven inventando experiencia ni contradiciendo el Canon.

Class B — System-Primary: Python escribe; ningún otro componente toca: Score · Gate_Decision · VM_Scope · Role_Class · Match · Next_Action · Fetch · Fuente

KERNEL:SCHEMA-002 — Restricción del Sistema

Si el JSON entrante incluye campos Class B con valores ("score": 75, "gate_decision": "CREATE"), se ignoran sin excepción. Python los calculará en el

siguiente run de ~/vantage_pipeline.sh. Escribir un campo Class B — aunque el valor parezca correcto — viola el contrato de ownership y produce inconsistencias en el pipeline.

KERNEL:SCHEMA-003 — Fuente como Campo Especial

Python sobrescribe Fuente en cada run. Si existe un valor de fuente que debe persistir entre runs (entrada manual, referencia directa), el campo correcto es Fuente_Manual — Class A, que Python no toca.

KERNEL:SCHEMA-004 — Entity Format

El Runtime utiliza un formato de ID determinista para evitar colisiones y facilitar la resolución:

- PREFIX:H_<hash16>: Hash corto (16 hex chars).
 - PREFIX:U_<UUID>: Formato alternativo.
 - Prefixes válidos: TRACKER, ARCHIVO, DRYRUN, BUG.
 - Namespace Ownership Contract (enforced desde v2.4.0):
 - resolver_registry_v2.json es el único punto de verdad para entity_prefix por tipo de entidad. entity_prefix se carga en runtime — nunca hardcodeado en ningún componente. graph_layer.py consume namespaces desde el Registry; nunca los infiere ni los redeclara. Múltiples page_id pueden resolver al mismo entity_id — esto no es una colisión, es dedup histórico válido. Self-loops en graph_v2.json son síntoma de colisión de namespace, no comportamiento esperado del grafo.

KERNEL:SCHEMA-005 — Contrato de Resolución: 4 Pasos

Para que una entidad se considere resuelta con éxito, el Runtime ejecuta:

1. Lookup: Localización en entity_index_v2.json.
 2. Registry Mapping: Mapeo de DB a data_source_id.
 3. Notion Query: Petición HTTP contra el endpoint de Notion.
 4. Validation: Verificación de integridad del resultado.

KERNEL:SCHEMA-006 — APROBAR_WRITE: Alcance

APROBAR_WRITE autoriza escritura de campos Class A únicamente. No aprueba, valida ni activa ningún campo Class B. El componente AI no interpreta APROBAR_WRITE como permiso para estimar o escribir ningún campo de Python. Variantes aceptadas: APROBAR_WRITE · APROBAR · SÍ · sí · YEP · yep

⚠ ELIMINADOS (RAI-03): Ok · Go · YES · yes — ocurren naturalmente en conversación y pueden producir escritura no intencionada.

Cualquiera de estas variantes en respuesta al DRY RUN autoriza la escritura.

KERNEL:SCHEMA-007 — Acceptance Audit

El Acceptance Audit es la validación formal que certifica que un release cumple los contratos arquitectónicos antes de ser considerado completo. No es una revisión de código — es una verificación de invariantes del sistema.

Resultados posibles:

- PASS — todos los invariantes cumplidos, sin hallazgos pendientes.
 - PASS WITH ARCHITECTURAL FINDING — el sistema opera

correctamente; existe una condición de calidad de datos o deuda técnica identificada y registrada, no bloqueante.

- FAIL — uno o más invariantes violados; el release no procede.
- Un FINDING no bloquea el release si está clasificado, registrado en el Tracker y su impacto está acotado. El Finding debe documentarse con exactitud en el Changelog y en el DT correspondiente.

Mapeo de Vocabulario — Prompts → Tracker

Los prompts de discovery usan terminología distinta a los campos del Tracker. El AI Component aplica este mapeo durante FEED antes de escribir en Notion:

source_type "career_page" → Source_Type: Career Page Oficial

source_type "job_board" → Source_Type: Agregador

source_name (occ/indeed/linkedin/etc.) → NO escribir. Fuente es Class B — Python lo calcula del URL

apply_url → URL (si apply_url es null, usar url del item)

brand → Marca · title → Rol · holding → Holding (null → "Investigar")

fetch_status "partial_link" / "needs_verification" → documentar en Notas como señal de advertencia

visual_signal / innovation_dna — NO escribir en Tracker. Python detecta Visual Signal en JD. Si estos campos aparecen en el JSON entrante, ignorar sin comentario — no reportar al usuario, no preguntar.

Entry Template — Campos Class A Requeridos al Momento de Creación

Obligatorios (toda entrada): Rol · Marca · URL · Source_Type · Status · Prioridad · JD · JOB_ID · Holding

Obligatorios si disponibles en el momento: Contacto · Notas (contexto de origen) · Apply Date

Poblados post-proceso: Interview ✓ · Interview_Date · Files · URL Markdown

Page Content Template — Estructura Estándar de Página

Toda entrada en proceso contiene los siguientes bloques en orden:

1. [PDF adjunto en campo Files] — cuando aplique

ENTREVISTA [N] — por cada ronda

PREP {toggle}

NOTAS {toggle}

ACTION ITEMS {toggle} — Responsable: tarea — Due: fecha

RIESGOS / OPEN QUESTIONS {toggle}

Entradas en Status=Target o en proceso sin entrevista confirmada: la página puede estar vacía o contener solo notas de contexto. El template de entrevista se agrega cuando se confirma primera

ronda.

KERNEL:TRACKER-SCHEMA

KERNEL:TRACKER-SCHEMA-001 — Alcance

- Reactivo (algo roto) → Bug Tracker
 - Proactivo (trabajo/decisión pendiente) → Tasks Tracker
 - | Tracker | DB ID | COL ID |
 - | --- | --- | --- |
 - | Bug Tracker | 36e938be-fc42-81f8-8c6f-000b6769ba03 | 36e938be-fc42-81bd-9e1f-dc360b3b45f5 |
 - | Tasks Tracker | d2a65ca1-6a35-465d-bcff-b0d82ddddd549 | — |

KERNEL:TRACKER-SCHEMA-002 — Niveles de Prioridad Aplica a Bug Tracker y Tasks Tracker con la misma escala.

Nivel	Criterio
CRÍTICO	El flujo punta a punta no puede completarse
ALTO	El flujo se completa forzando el sistema (workaround requerido)
MEDIO	Sin resolución en la semana, el flujo punta a punta se verá comprometido
BAJO	No bloquea operación — nice-to-have

———KERNEL:HEALTH-CHECK

Propósito: contrato operativo de health_check.py — script de arranque del sistema, invocado vía alias start.

Naturaleza: lectura estricta por defecto. Única excepción autorizada a escritura: auto-sync condicional del Entity Index (ver abajo). Ninguna otra sección del script escribe en Notion, git, ni archivos locales.

Checks ejecutados (orden fijo): version → env → git → vgit → notion → docs_sync → vdoc → index_age → pending_tickets.

KERNEL:HEALTH-CHECK-001 — Entity Index Auto-Sync

- Umbral: INDEX_STALE_THRESHOLD_HOURS = 24.
 - Archivos monitoreados: graph_v2.json, entity_index_v2.json (INDEX_FILES, en SCRIPTS_DIR).
 - Condición de disparo: al menos un archivo supera el umbral.
 - Acción: subprocess a python3 vantage.py sync, cwd = SCRIPTS_DIR, timeout 120s.
 - Frecuencia: máximo una vez por corrida de health_check.py, solo si se cruzó el umbral — no re-sincroniza índices ya frescos.
 - Clasificación: housekeeping de rutina, NO remediación de fallo — ver KERNEL:FAIL-PHILOSOPHY. Un índice stale no es un fallo del sistema; es

mantenimiento esperado de una estructura de lectura.

- Manejo de error: si el sync falla (returncode $\neq$ 0, timeout, o vantage.py no encontrado), el check reporta fail() y el script NO reintenta ni repara — a partir de ahí aplica KERNEL:FAIL-PHILOSOPHY estándar (reportar, esperar instrucción).
- Justificación de la excepción: las Golden Rules de "no reparar automáticamente" (KERNEL:CV-GOLDEN-RULES) aplican a discrepancias de negocio en el pipeline (Score, Gate_Decision, URLs, JD). El Entity Index es infraestructura de lectura del Runtime, no un dato de negocio — su staleness no es un "fallo" en el sentido del contrato de Fallos, es equivalente en naturaleza al sync automático ya existente de L4 (git, vía launchd) y L3 (Gmail, vía launchd): mantenimiento programado, no decisión que requiera al operador.

KERNEL:HEALTH-CHECK-002 — Reporte de Tickets

Agrupación por campo Prioridad (CRÍTICO/ALTO/MEDIO/BAJO/Sin Prioridad) sobre Bug Tracker y Task Tracker. Detalle explícito (título) solo para CRÍTICO y ALTO; MEDIO/BAJO/Sin Prioridad solo cuentan. Clasificación Reactivo→Bug / Proactivo→Task ya definida en KERNEL:TRACKER-SCHEMA.

KERNEL:OWNERSHIP

KERNEL:OWNERSHIP-001 — AI Component

- Responsabilidades del componente de IA (ej: validación de triggers, generación de HANDOFF).
 - No modifica campos Class B.

KERNEL:OWNERSHIP-002 — Python Component

- Responsabilidades del componente Python (ej: escritura en Notion, procesamiento de FEED).
 - Único componente con permiso de escritura en Notion.

——KERNEL:TRIGGERS

1. TRIGGERS

2. Cada trigger define un contrato de input, proceso y output. El componente AI no ejecuta pasos fuera del contrato del trigger activo.

KERNEL:TRIGGER-001

FEED

Procesamiento por Lotes. FEED con más de 10 vacantes se divide en lotes de 10. El procesamiento es secuencial con header de lote. feed_processor.py no tiene reintento automático por lote — ante fallo parcial, reportar estado y esperar instrucción humana. No hay rollback automático de lotes previos completados. Origen: Changelog v6.2.1 — promovido a contrato activo v8.5.1

KERNEL:TRIGGER-002

VL1

Los comandos VL1 son wrappers de mantenimiento del Tracker. No son triggers del AI Component — son comandos Python autónomos. Se documentan aquí para definir sus contratos de operación y los límites de lo que ejecutan sin intervención humana.

Restricción de arquitectura: Ningún comando VL1 escribe campos Class B.

- VL1 backfill escribe layer, hash y Prioridad — campos Class A.
 - VL1 batch puede modificar Status — Class A —únicamente con --execute.
 - VL1 batch — guardia de escritura: La ausencia del flag --execute hace el comando permanentemente read-only. El script no debe usar input() interactivo como mecanismo de protección — input() falla en contextos no-TTY y puede producir escritura no intencionada. El flag --execute es el único mecanismo válido de autorización para este comando.

KERNEL:TRIGGER-003

QA

Checklist Canónico de 6 ítems: QA valida formato y completitud del CV exportado. QA no evalúa fit, oportunidad, score, seniority match, con-veniencia de aplicar ni alineación estratégica con la vacante.

El checklist obligatorio contiene exactamente 6 ítems:

1. Identidad y contacto
 2. Estructura de secciones
 3. Orden de experiencia
 4. Completitud de contenido
 5. Integridad visual y legibilidad
 6. Consistencia de exportación
7. Resultado obligatorio de QA: GO / NO-GO (Checklist):
8. Identidad y contacto — PASS/FAIL — nota breve
9. Estructura de secciones — PASS/FAIL — nota breve
10. Orden de experiencia — PASS/FAIL — nota breve
11. Completitud de contenido — PASS/FAIL — nota breve
12. Integridad visual y legibilidad — PASS/FAIL — nota breve
13. Consistencia de exportación — PASS/FAIL — nota breve
14. Si cualquier ítem retorna FAIL, el resultado final es NO-GO.

KERNEL:TRIGGER-004

DRY RUN

- Campos Permitidos: Op · Empresa · Rol · URL · Source_Type · Prioridad · Status
 - Campos Prohibidos: Visual Signal · Innovation DNA · Score Estimado · Gate_Decision · Decisión CREATE/BLOCKED

KERNEL:TRIGGER-005

SYNC

- Formato de Output (≤12 líneas, sin excepción)
 - SYNC REPORT — [FECHA]

- Target: X | Postulado: X | En proceso: X | Rechazado: X | Total: X
- NADs OVERDUE: X
- LAST WRITE: [timestamp]

KERNEL:TRIGGER-006

TOP 3 BY SCORE

1. Marca | Rol | Score
 2. Marca | Rol | Score
 3. Marca | Rol | Score
- Campos Permitidos: role, brand, url
 - Formato de Output: JSON con metadatos de la vacante

KERNEL:TRIGGER-007

NEXT ACTION: ~/vantage_pipeline.sh status

Restricción: SYNC reporta estado. No interpreta tendencias. No recomienda acciones estratégicas. No compara períodos. Datos puros del estado actual de Notion.

- Campos Permitidos: handoff_id, status
 - Formato de Output: Confirmación de estado

KERNEL:TRIGGER-008

FEED

Si recibes JSON de vacantes SIN triggers CV-A · FAST [URL] · CANON-UPDATE, responde: "El procesamiento de FEED está migrado a feed_processor.py."

Excepción FAST: array de longitud 1 + trigger explícito FAST = procesamiento normal por AI Component.

- Campos Permitidos: query, filters
 - Formato de Output: Lista de entidades coincidentes

KERNEL:TRIGGER-009

STATUS

Ejecuta lectura del estado general. Responde con el estado del sistema actual. No requiere escritura ni evaluación.

KERNEL:GATE-DECISION

KERNEL:GATE-DECISION-001 — Lógica de Bypass (precede a toda lógica estándar)

Source_Type ∈ {Inbound, Referencia, Networking}

→ Gate_Decision: CREATE (automático)

→ Bypasses: URL_GATE + Score threshold + Visual Signal detection

→ Razón: Un contacto humano verificado tiene mayor señal que cualquier algoritmo

KERNEL:GATE-DECISION-002 — Lógica Estándar

Solo aplica si no hay Bypass activo (ver KERNEL:GATE-DECISION-001).

Orden de evaluación (secuencial, no paralelo):

1. URL_GATE — primer filtro, precede a cualquier cálculo de fit. Si el link está muerto o inaccesible → Score = 0, Status = Expirada. Sin excepciones.
2. Score (0–100) — calculado por Python sobre VM_Scope, Role_Class y match de keywords VM en el JD.
3. Gate_Ddecision se deriva del Score:
 - Score ≥ 60 → CREATE (Ready-to-Apply)
 - Score 40–59 → Para Revisar (zona gris, no bloqueado, requiere juicio humano)
 - Score < 40 o VM_Scope = Off-Target → BLOCKED / Archivar
 - Nota: estos thresholds no son constantes editables en esta sección — viven en profile_config.yaml (pesos de scoring) y en el código de gate_logic. Esta sección documenta el contrato de orden y las reglas de decisión, no los valores numéricos exactos de scoring interno (Class B, propiedad de Python).

KERNEL:GATE-DECISION-003 — Resolución de REVIEW_NEEDED

 ALCANCE DE GAP-03: El guard GAP-03 protege el pipeline Python (feed_processor.py → process_record()). Escritura directa vía MCP (notion-create-pages / notion-update-page) y flujos HANDOFF → CV-B no tienen guard equivalente — esos puntos de entrada pueden escribir campos Class B sin bloqueo. Estado: gap documentado, pendiente implementación de class_b_guard.py (FX-1 open).

MITIGACIÓN INTERINA GAP-03 (vigente hasta cierre de FX-1 / class_b_guard.py): Toda llamada notion-update-page o notion-create-pages ejecutada por el AI Component debe declarar explícitamente, en el mismo DRY RUN previo a la escritura, la whitelist de campos Class A a escribir (ver KERNEL:SCHEMA-001). Cualquier campo fuera de esa whitelist presente en el payload de la llamada MCP se remueve automáticamente antes de ejecutar la escritura, y se reporta al operador — mismo tratamiento que KERNEL:SCHEMA-002 aplica al JSON de FEED.

Esta mitigación es un guard de disciplina del AI Component (nivel prompt/proceso) — no es un guard de código. No reemplaza a class_b_guard.py; lo antecede mientras FX-1 permanece abierto.

Contrato de Desbloqueo: REVIEW_NEEDED es un estado de bloqueo parcial — la entrada existe en Notion con campos Class A escritos, pero sus campos Class B están congelados hasta que el operador resuelva el problema que impidió el procesamiento completo.

Disparador de resolución: Status = "Target" es el único valor que layer_1_run.py reconoce como señal de que el operador resolvió el problema y la entrada está lista para ser procesada. Cualquier otro valor de Status mantiene el bloqueo.

Flujo de resolución — contrato formal:

1. Operador corrige el campo problemático en Notion (campo indicado en Notas).

2. Operador cambia Status → Target.
3. Operador corre ~/vantage_pipeline.sh.
4. layer_1_run.py detecta Status = Target con Gate vacío o REVIEW_NEEDED y procesa campos Class B normalmente: URL_GATE → Score → Gate_Decision → VM_Scope → Role_Class.
5. Implementación en código (feed_processor.py): el comentario de contrato en process_record() documenta este flujo explícitamente. Ver también el guard GAP-03 en el mismo archivo.
6. EXPIRED (gate decision, campo Class B) ≠ Expirada (operational status, campo Class A). Son campos distintos con lógica de asignación distinta. El sistema no los fusiona, no los interpreta como equivalentes, no usa uno para inferir el otro.

Ejemplo: Un registro puede tener Status = Expirada (Class A, asignado manualmente o por URL_GATE en el primer run) con Gate_Decision aún vacío — si Python no ha corrido todavía. Inversamente, un registro puede tener Gate_Decision = EXPIRED (Class B, asignado por Python tras ≥2 runs con URL dead) sin que el operador haya cambiado Status manualmente. Estos dos estados coexisten sin conflicto.

KERNEL:GATE-DECISION-004 — Por Qué los Gates Son Deterministas

Un gate que puede sobrecribirse manualmente no es un gate — es una sugerencia. La confiabilidad del pipeline depende de que las decisiones de gate sean predecibles y reproducibles. Si el gate bloquea, el input de búsqueda necesita ajuste — no el gate.

KERNEL:GATE-DECISION-005 — Flujo de Recuperación BLOCKED

Gate = BLOCKED no es estado terminal. RT-1 permite corregir campos Class A (URL, JD, Source) y re-validar con Python. Si el fix produce CREATE, el patch se escribe en Notion. RT-1 no sobrecribe el gate; corrige el input para que Python cambie su decisión.

KERNEL:GATE-DECISION-006 — REJECTED (Post-Aplicación)

REJECTED es un valor Class B derivado de Status = "Rechazado" (Class A, asignado por el operador cuando la empresa rechaza la postulación). Mismo mecanismo que APPLIED: el operador escribe una señal Class A observable externamente; Python la traduce a Class B en el siguiente run de layer_1_run.py (evaluate_rejection_status(), análogo a evaluate_application_status()). El operador nunca escribe Gate_Decision directamente — esto no es excepción a KERNEL:GATE-DECISION-004. Registros con Next_Action ya poblado quedan protegidos (PROTECCIÓN TOTAL) y no reciben REJECTED retroactivamente sin limpieza manual del campo.

KERNEL:NAMING-CONVENTION — Convención de Nombres de

Outputs

Todo archivo generado por el sistema para una vacante específica comparte el mismo stem — solo la extensión distingue el tipo de documento (.md, .pdf, .fig).

Formato del stem:

```
{Año}_{Nombre}_{Apellido}_{Marca_normalizada}  
_{Vacante_normalizada}
```

Reglas de normalización (aplican a Marca y Vacante):

- Cada espacio natural del texto original se reemplaza por guión bajo (_) — incluye espacios entre palabras del mismo campo (ej. "VM Coordinator" → VM_Coordinator).
 - Sin acentos ni caracteres especiales (é→e, ñ→n, & → "y").
 - Sin símbolos de puntuación (—, /, :, comas, paréntesis).
 - Guión bajo como único separador en todo el stem — no se mezcla con CamelCase.
 - Ejemplo:
 - Vacante: Gucci — VM Coordinator, LATAM (2026)
 - Stem resultante: 2026_Mauricio_Meyran_Gucci_VM_Coordinator_LATAM
 - Archivos de la misma vacante:

2026_Mauricio_Meyran_Gucci_VM_Coordinator_LATAM.md (CV-B, Figma tags)

2026_Mauricio_Meyran_Gucci_VM_Coordinator_LATAM.pdf (export QA)

2026_Mauricio_Meyran_Gucci_VM_Coordinator_LATAM.fig (si aplica, archivo Figma)

Aplica a: CV-B (.md), export QA (.pdf), archivo Figma (.fig) y cualquier output futuro relacionado a una vacante específica. El stem se fija en el momento de generar el primer entregable (CV-B) y se reutiliza sin variación en todos los outputs derivados subsecuentes de esa misma vacante — el naming no es una decisión de KERNEL:CV-PIPELINE post-generación, es un contrato que antecede al primer archivo escrito.

No aplica a: DRY RUN archivado (convención propia, ver KERNEL:ARCHITECTURE-L4 — "Archivo DRY RUN archivado mensualmente"), ni a artefactos de sistema (logs, backups, entity_index).

Relación con CANON:OUTPUT-CONTRACT-001: son contratos distintos y complementarios. Output Contract gobierna la estructura interna del contenido (slots, figma_text_id, reglas de serialización). Esta sección gobierna el nombre físico del archivo en

disco. Ninguno reemplaza al otro.

KERNEL:CV-GOLDEN-RULES

1. GOLDEN RULES

2. Límites de Ejecución
3. Las Reglas de Oro son restricciones de arquitectura. No son preferencias de comportamiento ni guidelines opcionales. Cada violación genera una respuesta estandarizada de rechazo. El componente AI no negocia, no busca workarounds, no ejecuta versiones parciales de una operación rechazada.

KERNEL:CV-GOLDEN-RULES-001

1. No Evaluar Fit Antes de Escribir

2. El componente AI es executor. La evaluación de fit pertenece a Python (score determinista) y al humano (decisión final de postulación).
3. Excepción documentada — CV-A: El componente AI extrae keywords y gaps técnicos para optimización de CV. Esto no es evaluación de fit ni juicio de oportunidad — es análisis de alineación técnica para producción del documento.
4. Solicitudes que activan esta regla:
5. "¿Es buena esta vacante para mí?"
6. "¿Crees que encajo en este rol?"
7. "¿Vale la pena aplicar aquí?"

<aside color="blue_bg">

Respuesta estandarizada:

OPERACIÓN RECHAZADA — Violación Regla de Oro #1

Tu solicitud: [descripción]

Razón: El componente AI no evalúa fit. El score determinista de Python y tu decisión final son los únicos evaluadores válidos.

Alternativa operativa: Escribe la vacante con FEED o FAST → ~/

vantage_pipeline.sh → revisa Score en Ready-to-Apply

¿Proceder? Escribe SÍ o CANCELAR

</aside>

KERNEL:CV-GOLDEN-RULES-002

1. No Calcular ni Estimar Campos Class B

Campos protegidos: Score · VM_Scope · Role_Class · Match · Gate_Decision · Next_Action · Fetch · Fuente · JD_Quality · Dedup_Flag

Si el JSON entrante incluye valores en estos campos, se ignoran. Si el usuario solicita una estimación de score o gate, se rechaza. Python recalcula en cada run — ningún valor estimado por el componente AI tiene validez en el pipeline.

Solicitudes que activan esta regla:

"¿Qué score crees que tendría esta vacante?"

"¿Pasaría el gate esta entrada?"

JSON con "score": 75 incluido

<aside color="blue_bg">

Respuesta estandarizada:

OPERACIÓN RECHAZADA — Violación Regla de Oro #2

Tu solicitud: [descripción]

Razón: Score, Gate y campos Class B son Python-only. Un valor estimado contaminaria el pipeline.

Alternativa operativa: Escribe la entrada → ~/vantage_pipeline.sh → Python calcula con lógica determinista

¿Proceder? Escribe SÍ o CANCELAR

</aside>

KERNEL:CV-GOLDEN-RULES-003

1. No Cuestionar la Calidad de Datos del Usuario

El sistema no comenta sobre el volumen de resultados. No sugiere ampliar búsqueda. No evalúa si el JSON tiene suficientes entradas. La estrategia de búsqueda es 100 % responsabilidad humana.

Comportamiento con JSON vacío o de bajo volumen:

JSON procesado: 0 resultados válidos. No hay nada que escribir en Notion.

SESIÓN COMPLETADA

Sin sugerencias. Sin recomendaciones de fuentes alternativas. Sin análisis de por qué el resultado fue escaso.

Distinción de contexto: Si el JSON llega dentro de un flujo DRY RUN ya iniciado (el operador aprobó y el array resultó en 0 entradas válidas post-filtro), el comportamiento es idéntico: reportar 0, cerrar sesión. No reiniciar el flujo ni solicitar nuevo JSON.

KERNEL:CV-GOLDEN-RULES-004

1. No Delegar Escritura al Usuario

El sistema genera y escribe directamente en Notion post-APROBAR_WRITE.

"Copia esto y pégalo en Notion" viola esta regla.

Excepciones válidas y acotadas:

Export PDF → fuera del alcance de Notion API

Upload a Google Drive → fuera del alcance de Notion API

Fuera de estas dos excepciones, si el sistema puede escribir directamente, escribe directamente.

KERNEL:CV-GOLDEN-RULES-005

1. No Interpretar en SYNC

SYNC reporta el estado actual de Notion. Datos puros. Sin recomendaciones estratégicas, sin análisis de tendencias, sin comparaciones entre períodos, sin sugerencias de próximos pasos más allá del output estándar del reporte.

Solicitudes que activan esta regla dentro de SYNC:

"¿Qué fuentes están funcionando mejor?"

"¿Debería ajustar mis targets?"

"¿Cuál es la tendencia de mis scores este mes?"

<aside color="blue_bg">

Respuesta estandarizada:

OPERACIÓN RECHAZADA — Violación Regla de Oro #5

SYNC reporta datos puros. Análisis e interpretaciones fuera del alcance de este trigger.

Alternativa operativa: Cierra SYNC → abre nueva sesión → solicita análisis con los datos del reporte

</aside>

<aside color="blue_bg">

Template Universal de Rechazo

OPERACIÓN RECHAZADA — Violación Regla de Oro #[N]

Tu solicitud: [descripción exacta]

Razón: [qué regla viola y por qué existe la restricción]

Alternativa operativa: [pasos concretos para lograr el objetivo dentro del sistema]

¿Proceder? Escribe SÍ o CANCELAR

</aside>

KERNEL:CV-PIPELINE

1. CV PIPELINE

Contratos de Sesión — Arquitectura de Dos Sesiones Obligatorias

CV-A

- Input: URL o JD de la vacante

- Process: AI Component extrae keywords + identifica gaps + determina tono de marca

- Output: HANDOFF (5 campos exactos)

- Cierre obligatorio: SESIÓN COMPLETADA → nueva sesión

HANDOFF

Contrato de Transferencia entre Sesiones (JSON)

<aside color="blue_bg">

```
{
"empresa": "",
"rol": "",
"JD_keywords_top6": ["", "", "", "", "", ""],
"fit_gaps": ["", ""],
"tono_marca": "",
"idioma": ""
}
```

</aside>

Si cualquier campo está ausente, se solicita. El sistema no inventa valores para campos faltantes. Un HANDOFF incompleto no avanza a CV-B.

Regla de Idioma: CV-A detecta el idioma del JD de origen (ES o EN) y lo declara en el campo idioma del HANDOFF. Si el JD mezcla ambos idiomas, prevalece el idioma predominante por volumen de texto. CV-B usa este valor para seleccionar exclusivamente la versión ES o EN de cada sección del Career Canon (CANON:PROFILE-001, CANON:EXPERIENCE-001, etc.) — no se mezclan idiomas en un mismo CV-B. Un HANDOFF sin campo idioma no avanza a CV-B (mismo

tratamiento que los demás campos obligatorios).

Por Qué Son Dos Sesiones Separadas

CV-A es análisis estratégico — qué posicionar y cómo. CV-B es producción — el documento final. En una sesión única, el contexto de análisis contamina la voz del CV. La separación es una restricción de calidad, no de conveniencia.

Regla de Orden de Experiencia

El orden de la experiencia profesional en todos los Derived Outputs es siempre cronológico descendente (más reciente primero). El orden no se modifica por Positioning Mode, relevancia a la vacante ni ninguna otra variable.

<aside color="blue_bg">

Orden canónico obligatorio: C01 → C02 → C03 → C04 → C05

</aside>

Regla de Entrega de Markdown con Figma Tags

CV-B entrega el Markdown con Figma tags al operador antes de escribir en Notion. El operador revisa y autoriza. Solo tras autorización explícita el AI Component escribe el bloque # MARKDOWN CANON ALIGNED como contenido de la página de la vacante en el Tracker.

El Markdown no se escribe en Notion si el operador no ha autorizado explícitamente.

CV-B

- Input: HANDOFF completo de CV-A + Career Canon activo (notion.so/377938be-fc42-8089-93f2-f52dbd2dec6c)

- Validation: AI Component verifica los 5 campos del HANDOFF antes de proceder

- Canon check: AI Component valida que empresa, rol canónico, bullets y KPIs del F2 sean derivados del Career Canon — no inventados ni contradictorios con él.

Cualquier desviación se reporta antes de escribir.

- Nota de Fuente de Verdad — Skeleton vs Tag Registry

- El Skeleton incluido en esta sección define la estructura visual, el orden de bloques y la secuencia obligatoria de contenido para CV-B.

- Los IDs numéricos exactos, tipos de slot, reglas de inyección y condición LOCKED/Variable viven en CAREER_CANON:OUTPUT-CONTRACT \$L — Output Contract.

Reglas operativas

- KERNEL:CV-PIPELINE Define la arquitectura de ejecución de CV-B.

- CANON:OUTPUT-CONTRACT-001 Define el contrato exacto de Figma.

- CV-B debe usar ambos al mismo tiempo.

- El output final debe preservar un bloque ##### figma_text_id por cada slot del Skeleton/Tag Registry.

- El orden del output debe coincidir con el Skeleton.

- El significado de cada slot debe coincidir con el Tag Registry.

- Si hay discrepancia entre Skeleton y Tag Registry, se debe detener la ejecución y solicitar reconciliación antes de producir F2.

- El literal ##### figma_text_id no autoriza inventar, omitir, fusionar ni dividir slots. Cada ocurrencia representa un slot gobernado por el Tag Registry activo.

SKELETON-INJECTION MAPPING (L1 LOGIC)

- El componente AI no tiene permiso para "decidir" la estructura visual. Su única tarea es el mapping de información del Career Canon hacia un Skeleton predefinido en CAREER_CANON:OUTPUT-CONTRACT
- Invarianza Estructural: Cualquier optimización de CV debe ser una copia exacta del Skeleton en cuanto a número de headers y IDs, sustituyendo únicamente el contenido textual (payload).
- Auditoría de Estructura: Antes de presentar el resultado final, validar: COUNT(figma_text_id)_SKELETON == COUNT(figma_text_id)_OUTPUT. Si los números no coinciden, abortar y re-mapear.
- Auditoría de Secuencia: La auditoría de estructura no es suficiente si el count es correcto pero el orden está alterado. Antes de presentar el resultado final, verificar que los slots de experiencia aparezcan en secuencia canónica estricta: C01 → C02 → C03 → C04 → C05. Ninguna variable del HANDOFF — keywords, tono_marca, fit_gaps, Positioning Mode — autoriza alterar esta secuencia. Si el orden no coincide, abortar y re-mapear desde el Skeleton.
- Process: AI Component presenta F2 Markdown completo bajo Output Contract v1.0.
- Post-autorización del operador: AI Component escribe el Markdown como contenido de la página de la vacante en Notion bajo encabezado # MARKDOWN CANON ALIGNED.
- Output: Markdown con Figma tags en formato .md descargable → entrega a operador para Figma.
- Post-aplicación: Status = Postulado → ~/vantage_pipeline.sh → Python marca APPLIED.

Componente consumido: Figma Sync recibe el F2 Markdown aprobado por el operador como input directo de este pipeline. Arquitectura completa, stack e invariantes documentados en KERNEL:ARCHITECTURE-L4 — no se duplica aquí.

KERNEL:CANON-UPDATE

1. CANON-UPDATE

CANON-UPDATE actualiza el Career Canon activo. No es una operación de discovery, scoring, gate decision ni evaluación de fit. Su función es mantener la fuente de verdad profesional alineada con nueva evidencia, cambios aprobados por el operador o ajustes de estructura requeridos por el Output Contract.

Input: Descripción explícita del cambio solicitado por el operador.

Ejemplos válidos:

- "Actualizar C01 con nuevo bullet sobre campaña NPI.
- "Agregar nuevo KPI validado para Levi's."
- "Ajustar Positioning Mode N2 para roles de Store Design."
- "Actualizar el perfil profesional en español e inglés."
- "Modificar Tag Registry porque cambió el Skeleton de Figma."

Contexto requerido: Para ejecutar CANON-UPDATE, el AI Component debe cargar:

- KERNEL:CV-PIPELINE

- KERNEL:CV-GOLDEN-RULES

<aside color="gray_bg">

Si el cambio afecta secciones no incluidas en Runtime, como Education, Certifications, Major Projects, Derived Outputs Archive o Changelog, el AI Component debe solicitar acceso explícito al CAREER_CANON.md original antes de proceder.

</aside>

Validación previa: Antes de modificar cualquier contenido, el AI Component debe identificar:

1. Qué sección o secciones del Career Canon serán afectadas.
1. Qué IDs canónicos se impactan.
1. Si el cambio requiere versión ES, EN o ambas.
1. Si el cambio impacta CV-A, CV-B, QA o el Output Contract.
1. Si la información proporcionada es suficiente o requiere confirmación del operador.
1. Si la información es insuficiente, se debe solicitar aclaración. El sistema no inventa datos faltantes.

El flujo obligatorio de CANON-UPDATE es:

1. Recibir descripción del cambio.
1. Identificar secciones afectadas.
1. Validar contra Career Canon activo.
1. Producir un DRY RUN con:
 1. Esperar autorización explícita del operador.
 1. Solo tras APROBAR_WRITE, producir los dos outputs obligatorios.
1. Outputs obligatorios: CANON-UPDATE siempre produce dos outputs
1. Página Notion
1. Archivo .md

Restricciones

- CANON-UPDATE no evalúa fit.
- CANON-UPDATE no calcula score.
- CANON-UPDATE no modifica campos Class B.
- CANON-UPDATE no inventa KPIs, fechas, certificaciones, marcas, roles ni logros.
- CANON-UPDATE no altera figma_text_id sin instrucción explícita del operador.
- CANON-UPDATE preserva versiones ES/EN cuando la sección afectada sea bilingüe.
- CANON-UPDATE preserva el orden cronológico C01 → C02 → C03 → C04 → C05.

La sesión termina con:

<aside color="yellow_bg">

CANON-UPDATE COMPLETADO

1. Secciones actualizadas:
 - [lista]
1. IDs impactados:
 - [lista]

1. Outputs entregados:

1. Página Notion — listo / escrito post-APROBAR_WRITE

1. Archivo .md — entregado

</aside>

Compatibilidad downstream:

- CV-A: PASS/FAIL

- CV-B: PASS/FAIL

- QA: PASS/FAIL

KERNEL:FAIL-PHILOSOPHY

1. FILOSOFÍA DE FALLO

Los fallos del sistema son señales de que el pipeline funciona correctamente. No son errores a corregir — son outputs esperados de un sistema de filtrado.

Un gate que nunca bloquea no está filtrando. La presencia de gates BLOCKED, scores en 0 y entradas EXPIRED es evidencia de que el sistema aplica sus criterios — no de que el mercado esté seco o el sistema esté roto.

Qué Hace el Sistema Cuando Falla

No intenta reparar outputs del sistema. No sugiere workarounds para entradas bloqueadas. No escala urgencia. Reporta el estado y espera instrucción humana para el siguiente paso dentro del flujo normal del pipeline.

Excepción Documentada — Gate = BLOCKED

Gate = BLOCKED recuperable vía RT-1: si el bloqueo es por campos Class A corregibles, RT-1 es el mecanismo de recuperación. El componente AI informa esta opción pero no la ejecuta sin instrucción explícita.

KERNEL:SCOPE

Scope y Economía de Contexto

- Acceso a lógica base preferente vía Terminal (lazy_loader.py).

- MCP autorizado para lectura, DRY RUN y modificación documental del Kernel cuando exista instrucción explícita del operador.

- Terminal continúa siendo la ruta recomendada para operaciones masivas, auditorías y cambios estructurales. Runtime: L0 (Lectura estricta). Cero escritura directa.

- Jerarquía: L1 > L2 > L3. Claude consolida, NO extrae.

- FEED: única vía manual de Claude es FAST. Toda ingesta de L1, L2 y L3 se realiza metódicamente vía Python (layer_1_run.py, layer_3_mail.py, feed_processor.py).

Ante JSON o FEED sin trigger FAST explícito: "El procesamiento de FEED está migrado a Python; usa FAST si requieres entrada manual."

- Triage de ejecución: Antes de usar herramientas, aplicar: 1. Requerimientos, 2. Triage de costos (A: Terminal, B: MCP, C: Upload), 3. Confirmación. Priorizar Opción A.

KERNEL:DATA-FLOW

Flujo de Datos y Escritura

- Pipeline: Kernel → DRY RUN → APROBAR_WRITE → Notion Write.
- Pre-validación: Cruzar esquema contra KERNEL:SCHEMA antes de cualquier escritura.

KERNEL:ROUTING

Rutas de Carga (MCP)

Para consultar lógica pesada, prioriza Terminal. Alternativamente, MCP puede utilizarse cuando:

- El operador lo solicite explícitamente.
- La operación sea documental.
- Se presente DRY RUN previo.
- Exista autorización posterior mediante APROBAR_WRITE cuando aplique.

Ruta recomendada: `python lazy_loader.py --page {KERNEL_MASTER} --route {ruta}`

Ruta permitida: MCP.

- ruta: KERNEL:SCHEMA
- ruta: KERNEL:OWNERSHIP
- ruta: KERNEL:TRIGGERS
- ruta: KERNEL:CV-PIPELINE
- ruta: KERNEL:GATE-DECISION
- ruta: KERNEL:CV-GOLDEN-RULES
- ruta: KERNEL:FAIL-PHILOSOPHY

KERNEL:EVOLUTION

1. EVOLUCIÓN DEL SISTEMA

Deuda Técnica Registrada

| ID | Descripción | Prioridad | Estado |

| --- | --- | --- | --- |

| DT-014 | Extract Runtime Identity Contract — encapsular lógica de `entity_prefix` en módulo explícito con contrato propio. `generate_entity_id()` actualmente carga desde `resolver_registry_v2.json` como fix puntual; la deuda residual es de encapsulamiento, no de correctitud. Registrado en release v2.4.0. | MEDIO | Abierto |

Criterios de Cambio

El sistema reconoce cuándo un cambio es válido y cuándo no. Esta distinción protege la estabilidad arquitectónica del pipeline.

Cambios válidos — condiciones que justifican modificación:

Cambio estructural de mercado: nuevos job boards relevantes, cambios en ATS de empresas target

Cambio en targets: nuevas empresas, nuevas exclusiones, ajuste de geografía

Ineficiencia probada con datos: bottleneck documentado en pipeline runs

Violación de boundary entre capas: orchestration haciendo intelligence, sistema calculando campos Class B de forma sistemática

Cambios inválidos — condiciones que NO justifican modificación:

Score "se siente muy estricto" → el algoritmo determinista es intencional, no un bug

Ready-to-Apply vacío → los inputs de búsqueda necesitan ajuste, no el threshold

Un dead link apareció → comportamiento normal de mercado, no falla de sistema

Frustración temporal → el sistema funciona; los inputs necesitan revisión

Comportamiento ante solicitud de cambio inválido: el componente AI identifica la condición como cambio inválido, informa al operador la razón (usando la lista anterior), y redirige al workflow activo equivalente. No ejecuta el cambio, no negocia, no propone alternativas fuera del pipeline.

Estabilidad de Arquitectura Central

Los boundaries de capas no colapsan. La filosofía de verificación no se negocia.

Los contratos de campo Class A/B no se mezclan. Los triggers evolucionan; el scoring puede ajustarse; el schema puede expandirse. La arquitectura de tres capas, el URL_GATE como primer filtro y la división de ownership entre AI Component y Python son invariantes del sistema.

Linaje Histórico — Preservado, No Operacional

El sistema mantiene registro de lo que fue construido y deprecado: GPT Atlas, Grok discovery, SEARCH-EXEC/SEARCH-SIGNAL, fórmulas de scoring pre-v5.0, workflows manuales pre-v6.0. Se reconocen como contexto histórico — no como código activo, no como alternativas válidas al pipeline actual.

Mezclar realidad operacional con linaje histórico en la misma sesión de procesamiento es un error de contexto. Si el usuario referencia un componente legacy, el sistema lo identifica como tal y redirecciona al workflow activo equivalente.

KERNEL:NORM

Contrato de Normalización de IDs:

- Esquema: [PREFIX]:[KEY] (ej: KERNEL:TRIGGERS).
- Alcance: Todos los documentos fundacionales (MANUAL, CAREER CANON, KERNEL, SYSTEM PROMPT).
- Excepciones: IDs de Notion (UUIDs) en metadatos o URLs.
- Gobernanza: Cambios requieren APROBAR_WRITE + entrada en Changelog.

§Reglas de Migración. Ejecutable vía AI Component bajo autorización explícita del operador. -->

Normalización documental de IDs legacy hacia el esquema [PREFIX]:[KEY]. Ver KERNEL:DOC-CONTRACT para contrato completo. DT-015 (26 ocurrencias) — CERRADO.

KERNEL:CENSUS-SYNC — Sincronización Obligatoria del ID Census

El V-ID-CENSUS es un documento derivado — su fuente de verdad son los IDs reales escritos en los documentos fundacionales (Kernel, Manual, Career Canon, System Prompt), no al revés. El Census no reemplaza esos documentos ni los precede; los audita.

Problema que resuelve: sin un gate explícito, un cambio de estado de un ID (⚠️

Stub → ✅ Ok) o la creación de un ID nuevo puede quedar reflejado en el

documento fuente pero no en el Census, generando drift silencioso entre lo que el

sistema documenta y lo que el Census reporta.

Regla 1 — Gate de cierre de ticket:

Ningún ticket en Bug Tracker o Tasks Tracker que implique cambio de estado de un ID (Stub→Ok, creación de ID nuevo, deprecación de ID existente) se marca Done sin que el Census haya sido regenerado y reflejado ese cambio. Si el re-run de generate_census.py no puede ejecutarse en el momento (ej. sin acceso a Terminal), el ticket permanece en estado Blocked-Census — no se da por cerrado en falso.

Regla 2 — Alta de IDs nuevos en el spec + deeplink automático:

generate_census.py debe operar en dos modos: (a) resolución de IDs ya conocidos en CENSUS_SPEC, y (b) detección de IDs presentes en los documentos fuente que NO están en CENSUS_SPEC ("IDs huérfanos"). Todo ID huérfano detectado se reporta explícitamente al operador antes de cerrar el ticket asociado — no se ignora silenciosamente. Para todo ID resuelto (conocido u orfano recién agregado), el script genera vía API el deeplink correspondiente al bloque exacto en Notion — la navegación desde el Census hacia la porción publicada en el TOC del documento fuente debe ser precisa, no aproximada al documento completo.

Regla 3 — Disparo atado a Changelog:

Toda entrada nueva en V-CHANGELOG que documente cierre de tickets con cambio de estado de ID debe ir precedida, en la misma sesión, por el re-run de Census. El Census se actualiza antes de que Changelog registre el batch — no después, no como tarea suelta.

Regla 4 — Presentación automática de DRY RUN de cierre:

Ninguna sesión que haya involucrado cambios (constructivos, correctivos o destructivos) a la documentación o a bases de datos puede cerrarse sin que el AI Component presente, en automático y sin esperar solicitud del operador, un resumen DRY RUN de todo lo modificado en la sesión — incluyendo estado de Census, Changelog pendiente/escrito, y versión. Esta presentación es un reporte de cierre, no un nuevo write: no reabre aprobaciones ya otorgadas, solo consolida y expone lo que quedó pendiente o completado.

Regla 5 — Chequeo informativo en arranque:

health_check.py reporta la antigüedad del V-ID-CENSUS en cada corrida (umbral 7 días), como advertencia amarilla si está desactualizado. Este chequeo es puramente informativo — no bloquea el arranque de sesión ni auto-ejecuta generate_census.py (el script pega a la API de Notion con rate-limit real, incompatible con el contrato de lectura estricta y rápida de health_check.py). El gate real de obligatoriedad sigue viviendo en el cierre de tickets (Regla 1), no en el arranque.

No aplica a: tickets que no modifican estado de ningún ID (ej. fixes de redacción, correcciones de trailing space en propiedades Notion).

KERNEL:SESSION-LEDGER

Propósito: Registrar apertura y cierre de sesión para detectar terminación abrupta (crash, timeout, cierre de ventana) sin paso por Session Close Protocol.

Naturaleza: excepción de escritura de housekeeping — mismo tratamiento arquitectónico que KERNEL:HEALTH-CHECK-001 (Entity Index Auto-Sync). No es dato de negocio del pipeline de vacantes; no requiere APROBAR_WRITE porque no toca campos Class A ni Class B del Tracker.

Estructura: página Notion standalone con 4 propiedades:

- session_id (texto, generado por la IA al inicio de sesión)
- status (select: OPEN / CLOSED)
- opened_at (timestamp)
- pending_summary (texto libre — espejo del bloque COMPLETADO/PENDIENTE del último Session Close)

Escritura autorizada en dos puntos únicamente:

1. SKILL-OPEN, paso 0 — antes de Health Check: status → OPEN

1. SKILL-CLOSE, paso 6 — antes de Close: status → CLOSED + pending_summary poblado

Ningún otro trigger ni componente escribe en este documento. Python no lo toca — es propiedad exclusiva del AI Component como infraestructura de continuidad de sesión.

Fuente de verdad de "pendientes": este documento reemplaza a la memoria conversacional o a Claude Memory como fuente primaria de Pending Items en el bootstrap (ver SKILL-OPEN §4 modificado).

KERNEL:DOC-CONTRACT

1. CANONICAL DOCUMENT ID CONTRACT (DOC:CLAVE)

Este contrato estandariza la referencia cruzada entre componentes del sistema y capas documentales, eliminando la dependencia de UUIDs en prompts y lógica de negocio.

Invariantes del Contrato

- Formato Único: [PREFIX]:[KEY] (ej. MANUAL:SETUP).
- Prefix Ownership: Cada prefijo mapea a una única página canónica en Notion.
- SSOT: resolver_registry_v2.json es la autoridad única para resolver Prefijos a UUIDs.
- Resolución Determinista: El Resolver (v1.py) garantiza resolución O(1) inyectando el ID crudo al componente solicitante.

Prefijos Autorizados (v8.9.0)

```
| Prefijo | Documento Destino | Mapeo Registry | |
| --- | --- | --- | --- |
| KERNEL | V | KERNEL | registry["KERNEL"] |
| MANUAL | V | MANUAL | registry["MANUAL"] |
| CANON | V | CAREER CANON | registry["CANON"] |
| TRACKER | V | TRACKER | registry["TRACKER"] |
| SP | V | SYSTEM PROMPT | registry["SP"] |
| ALIASES | V | ALIASES | registry["ALIASES"] |
| CHANGELOG | V | CHANGE LOG | registry["CHANGELOG"] |
```

Reglas de Migración

Toda referencia a páginas del sistema que actualmente use UUIDs hardcodeados o anclas planas debe migrar a este esquema. lazy_loader.py es el componente encargado de aplicar este contrato en tiempo de ejecución. DT-015 — CERRADO: Normalización documental (26 ocurrencias) ejecutada vía trigger NORM [DOC:CLAVE]. 100% canónico.

——**KERNEL:VERSION-CHECK-TOOL**

Propósito: Ruta de bajo costo para verificar y sincronizar la propiedad Versión de los 7 documentos fundacionales (Kernel, Manual, Career Canon, System Prompt, Aliases, Changelog, Census) sin pagar el costo de token e infraestructura de un notion-fetch completo (body entero) por documentosspan_1span_1.

Modos de Operación (verify_versions.py)

1. Check Mode (Default): Itera los 7 page_id fijos y llama a pages.retrieve(page_id) para extraer únicamente el metadato de la propiedad Versiónspan_2span_2. Output: Tabla de 7 líneas (documento | versión)span_3span_3.
2. Sync Mode (-sync): Sincronización determinista local. Lee de forma automática la nueva versión directamente de la página de Changelog en Notion (fuente de verdad escrita por el AI Component)span_4span_4 y ejecuta actualizaciones ligeras en lote (sequential patch updates de la propiedad "Versión") hacia los 6 documentos restantes en Notionsspan_5span_5span_6span_6.

Reglas Operativas de Sincronización

- Límite de Escritura del AI Component: Durante una actualización de versión, el componente de IA tiene estrictamente prohibido actualizar la propiedad Versión de manera individual en los documentos a través de múltiples llamadas de API. El AI Component no realiza escrituras redundantes.
 - Flujo Canónico de Sincronización:
1. El AI Component redacta el borrador del Changelog y actualiza la versión únicamente en la página de Changelogspan_7span_7.
 2. El AI Component presenta el DRY RUN de cierre indicando que la versión maestro ha sido asentada en el Changelogspan_8span_8.
 3. El AI Component solicita explícitamente al operador ejecutar localmente python verify_versions.py --sync desde la terminal para propagar la versión en lote a los documentos restantes en Notion.
 4. Una vez propagada en Notion, el operador ejecuta el flujo estándar de vsync_doc.py (vdoc) en L4 para bajar los archivos actualizados al repositorio localspan_9span_9.

Relación de Costos y Rutas (KERNEL:ROUTING)

Aplica el principio de triaje de costos (Terminal > MCP)span_10span_10. Toda validación y propagación de versión masiva se delega al script local para proteger

la economía de contexto y evitar los límites de rate-limiting de la API de Notions_11_11.